

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM

Khoa Công Nghệ Điện Tử - Bộ môn Điện Tử Máy Tính

BÀI GIẢNG

**HỆ THỐNG NHÚNG
(Embedded System)**

TP.HCM-2024

NỘI DUNG MÔN HỌC

Chương 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Chương 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

Chương 3: CẤU TRÚC PHẦN MỀM

Chương 4: THIẾT KẾ HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

1. HỆ THỐNG NHÚNG
2. PHẦN CỨNG HỆ THỐNG NHÚNG
- 3. PHẦN MỀM HỆ THỐNG NHÚNG**
4. CÔNG NGHỆ Multi-Core, SoC, and FPGA
5. MỘT SỐ ỨNG DỤNG HỆ THỐNG NHÚNG

3. PHẦN MỀM HỆ THỐNG NHÚNG

Contents

- ☐ Device Drivers
- ☐ Embedded Operating Systems
- ☐ Middleware
- ☐ Application Software

3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Basically, embedded software can be divided into two general classes

- **Systems software.** Supports the applications, includes:
 - 1) **Device Drivers**; 2) **Operating Systems** and 3) **Middleware**.
 - **Application software:** is the *upper-level software* that defines the function and purpose of the embedded device and which handles most of the interaction with *users* and *administrators*.
- All embedded systems that require software have, at the very least, **device driver software** in their **system software layer**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

→ Most embedded hardware requires some type of software initialization and management:

- ✓ The **software** that directly interfaces with and controls this **hardware** is call a ***device driver***.
- ✓ **Device drivers** are the **software libraries** that initialize the hardware, and manage access to the hardware by higher layers of software.
- ✓ **Device drivers** are the liaison between the hardware and the ***operating system, middleware, and application layers***.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

- ✓ **Device drivers** are typically considered: **Architecture-specific** or **Generic**.
 - + **Architecture-specific drivers** initialize and enable components **within a master processor** include **on-chip** memory, integrated memory managers.
 - + **Generic driver** manages hardware that is located **on the board** and not integrated onto the master processor: **off-chip memory** (*controllers, level-2+ cache, Flash, etc.*), **off-chip I/O** (*Ethernet, RS-232, display, mouse, etc.*), **board buses** (*I2C, PCI, CAN, etc.*).
- ✓ Each of driver function typically has code that interfaces directly **to the hardware** and code that **interfaces to higher layers of software**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ Device drivers for **Interrupt-Handling**

Interrupts are signals (*triggered by events*) that cause the master processor to **stop executing** the current instruction stream and **start *the interrupt (IR) process***.

- ✓ **Software interrupts** are explicitly triggered internally by some instruction within the current instruction stream being executed **by the master processor**.
- ✓ **Internal hardware interrupts** are initiated by an event that is **a result of a problem** with the current instruction stream that is being executed by the master processor: **illegal math operations** (*overflow, divide-by-zero*), **debugging** (*single-stepping, breakpoints*), **invalid instruction**,...
- ✓ **External hardware interrupts** are initiated by hardware other.

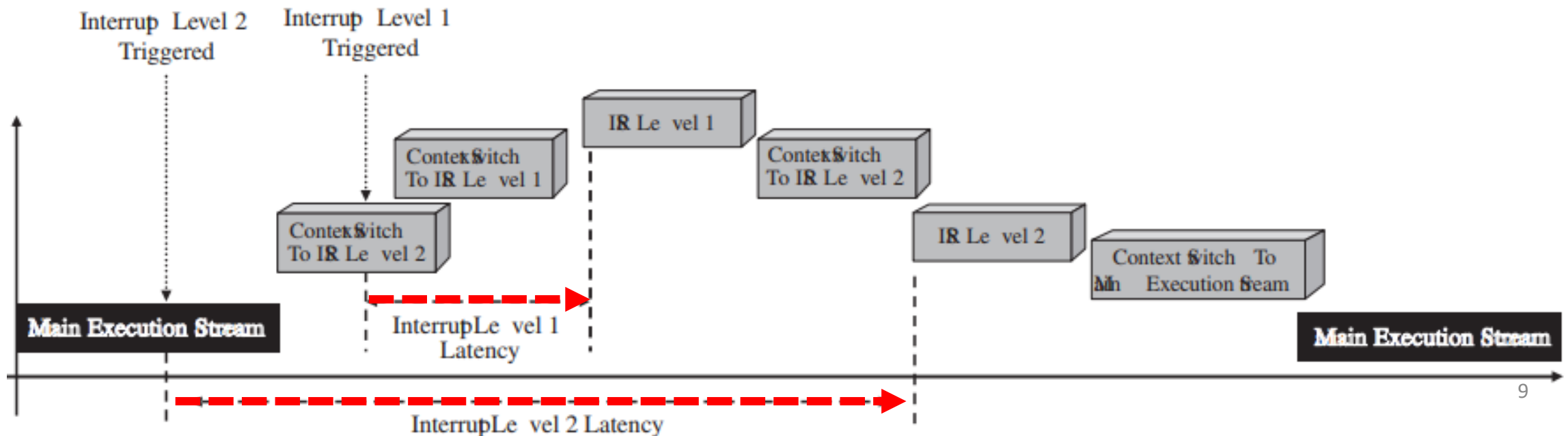
3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ Device drivers for **Interrupt-Handling**

The **performance** of an embedded design is affected by the **latencies** (delays) involved with the interrupt-handling scheme. The figure summarizes the variables that impact **interrupt latency**.



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ Memory Device Drivers

- + The software must **provide the processors** in the system with the ability to access various portions of the memory map.
- + The software involved in **managing the memory on the master processor and on the board**, as well as managing memory hardware mechanisms, consists of the **device drivers** for the management of the overall **memory subsystem**.
- + The **memory subsystem** includes all types of memory management components, such as **memory controllers** and **MMU**, as well as the types of memory in the memory map, such as **registers, cache, ROM, RAM, ...**

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ On-board Bus Device Drivers

- + Some type of **protocol** that defines how devices gain **access to the bus**.
- + The **rules** attached devices must follow **to communicate over the bus**.
- + The **signals** associated with the **various bus lines**
- + **An example (I2C protocol)**

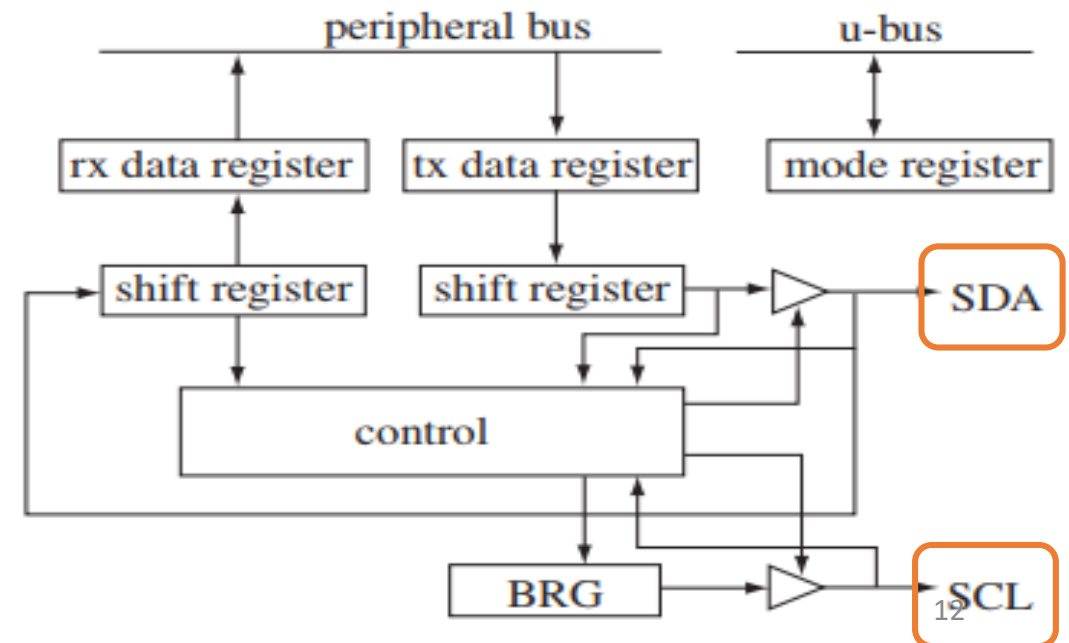
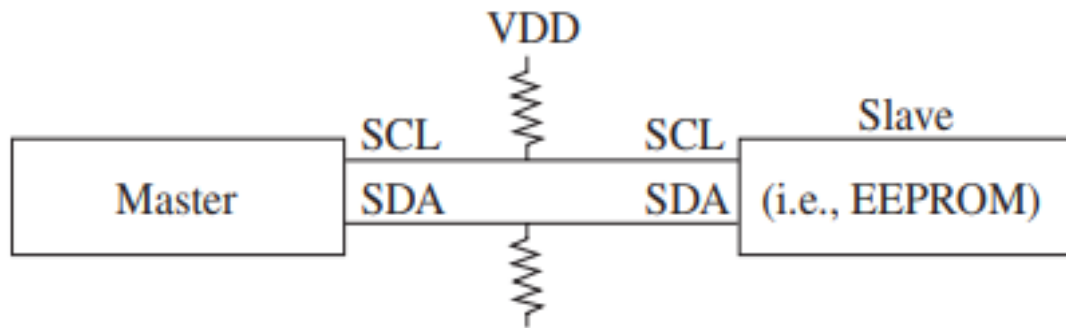
3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ On-board Bus Device Drivers

I2C protocol (*Inter-Integrated Circuit*) is a serial bus with one serial data line (**SDA**-Serial Data) and one serial clock line (**SCL**-Serial Clock).



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ Board I/O Drivers

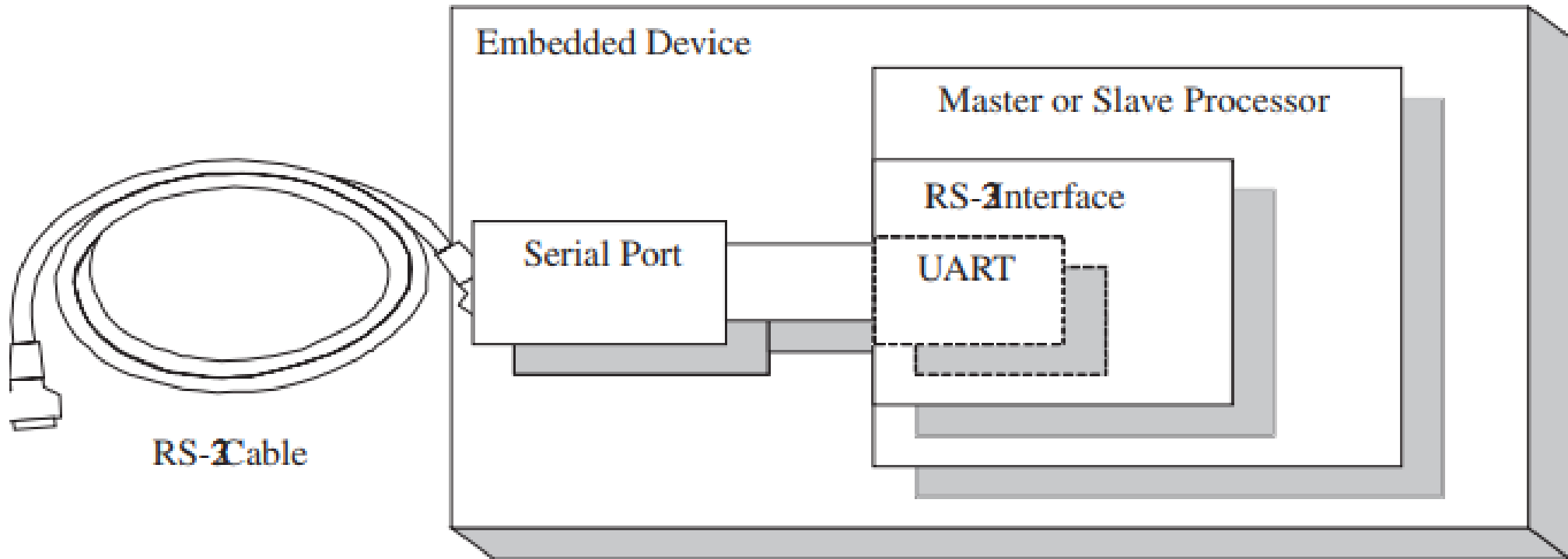
- + The board I/O subsystem components that require some form of **software management** include the components integrated on the **master processor**, as well as an **I/O slave controller**, if one exists.
- + The **I/O controllers** have a set of status and control registers used to control the processor and check on its status.
- + **An example (Driver RS-232)**

3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.1 Device Drivers

Driver RS-232



3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

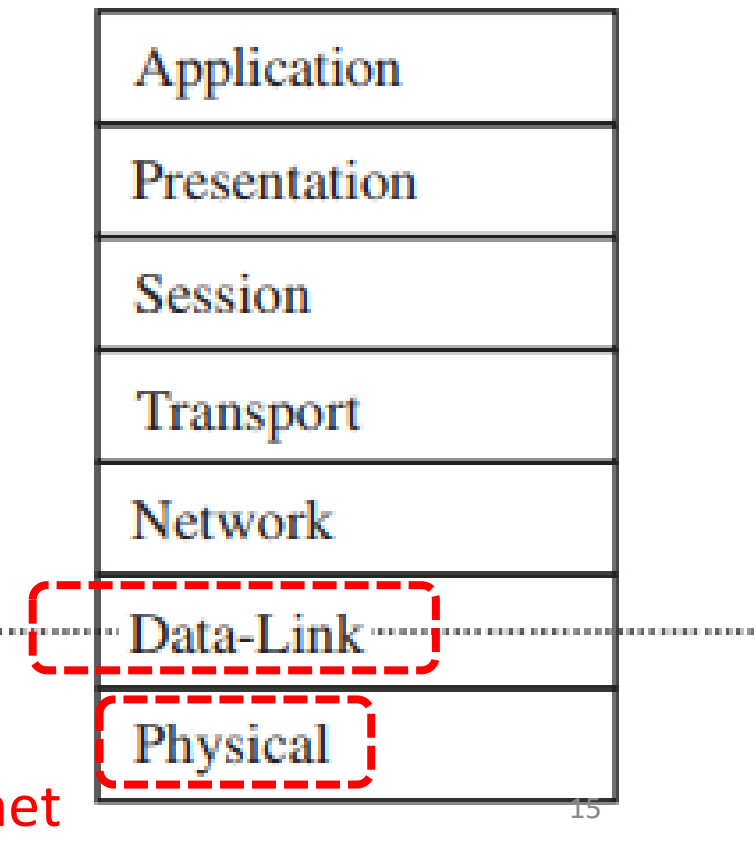
3.1 Device Drivers

Driver **RS-232**, *asynchronous serial I/O* protocol:

OSI model (as Figure): is a reference tool to use in understanding *real-world networking protocols* implemented in embedded devices.

+ For **Driver RS-232**, the **firmware** required to enable RS-232 functionality maps to the lower section of the *Data-link layer* (in **OSI** model).

+ The **hardware components** can all be mapped to the *Physical layer* of the **OSI** model.



3. PHẦN MỀM HỆ THỐNG NHÚNG

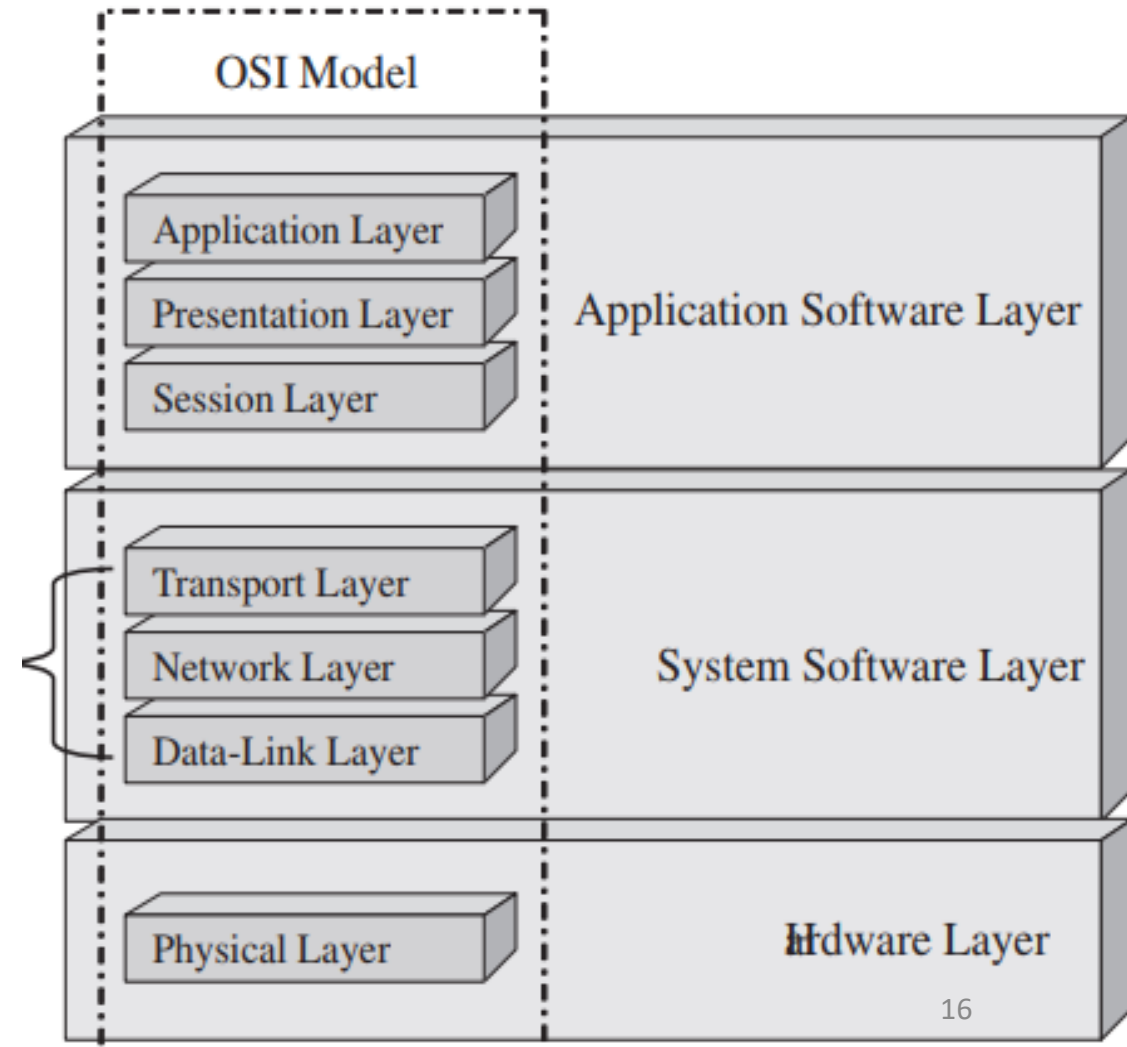
CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ **OSI model** (Open Systems Interconnection)

+ The OSI model **is created** in the early 1980s by the International Organization for Standardization (**ISO**).

+ The OSI model **represents** the required hardware and software components within **a networked device** in the form of **seven layers**: *physical*, *data-link*, *network*, *transport*, *session*, *presentation*, and *application layers*.



3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ OSI model

+ In OSI model, the data is sent over the transmission **to the physical layer** of a connected device, and then up through the connected **device's layers**.

+ The functionality and methodologies implemented at each layer based on the **OSI** model are **also commonly referred** to as **networking protocols**.

Application	End-use interface
Presentation	Data format
Session	Application dialog control
Transport	Connections
Network	End-to-end service
Data link	Reliable data transport
Physical	Mechanical, electrical

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

❑ OSI model

- + **Physical layer**: defines the basic properties of the interface between systems, including the *physical connections* (plugs and wires), *electrical properties*, *basic functions* of the *electrical and physical components*, and the *basic procedures* for *exchanging bits*.
- + **Data link**: Detect *error* and *control* across a single link.
- + **Network**: Define the basic *end-to-end data transmission service*.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.1 Device Drivers

- + **Transport:** Define connection-oriented services that *ensure that data* are delivered in the *proper order* and *without errors* across multiple links. This layer may also try *to optimize* network resource utilization.
- + **Session:** Provide mechanisms for *controlling the interaction* of *end-user services across a network*, such as data grouping and checkpointing.
- + **Presentation:** Define *data exchange formats* and *provides transformation utilities* to application programs.
- + **Application:** Provide the *application interface* between the *network* and *end-user programs*.

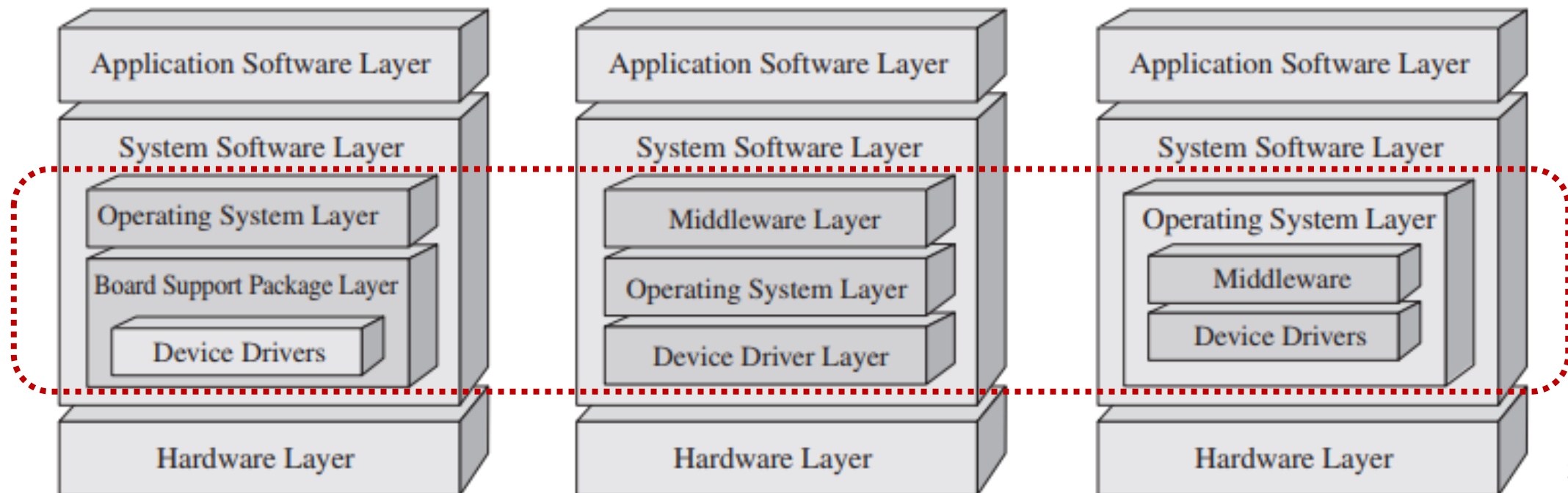
3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

+ An **operating system (OS)** is an optional part of an embedded device's system software stack.



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

- + The **OS** is a set of software libraries that serves two main purposes:
 - ✓ Providing an abstraction layer for software on top of the OS (be less dependent on hardware) → Making the development of **Middleware** and **Applications** that sit on top of the OS easier.
 - ✓ Managing the various system hardware and software resources → **Ensure** the entire system **operates efficiently** and **reliably**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

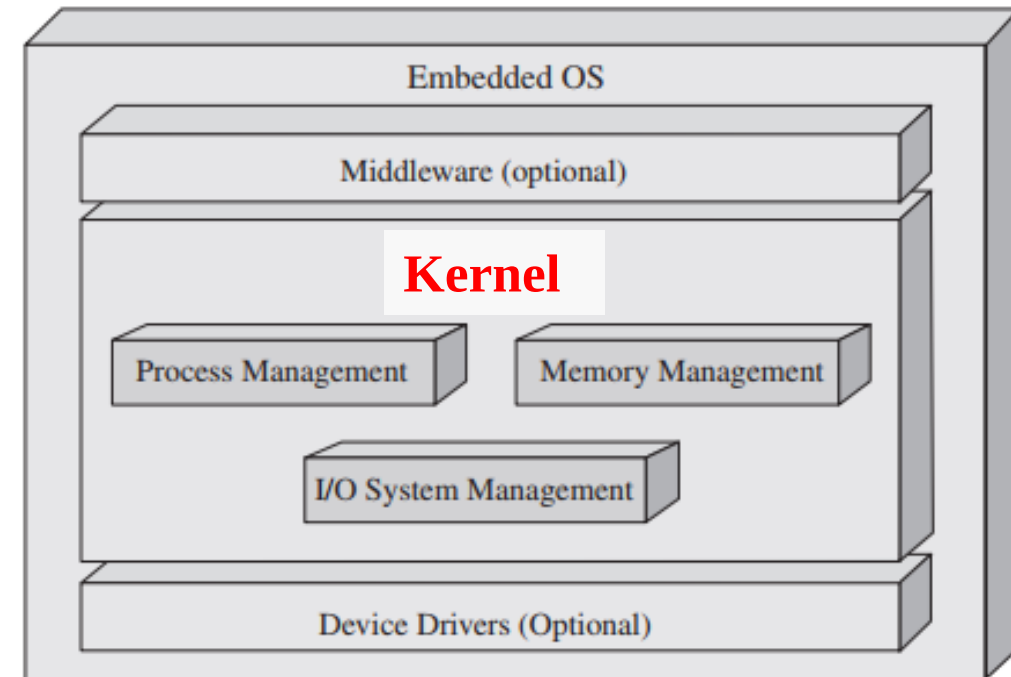
3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

+ An **OS** is composed of multiple software components, and the core components in the OS form its **kernel**.

+ The **kernel** always runs in *system mode*, while other parts and all applications run in *user mode*.

+ All **OSes** have a **kernel** at the very least. The **kernel** is a component that contains *the main functionality* of the OS (**Figure**).



- Process Management
- Memory Management
- I/O System Management

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

There are some of the **main functions** of a typical general-purpose OS:

1) Process Management

- A **process** is an instance of a program in execution. It is a unit of work within the system. Starting a new process is a heavy job for the OS, which includes allocating memory, creating data structures, and coping code.
- Sometimes, processes need to communicate with each other while they are running. This is called **Inter Process Communication (IPC)**. The OS provides mechanisms to support **IPC**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

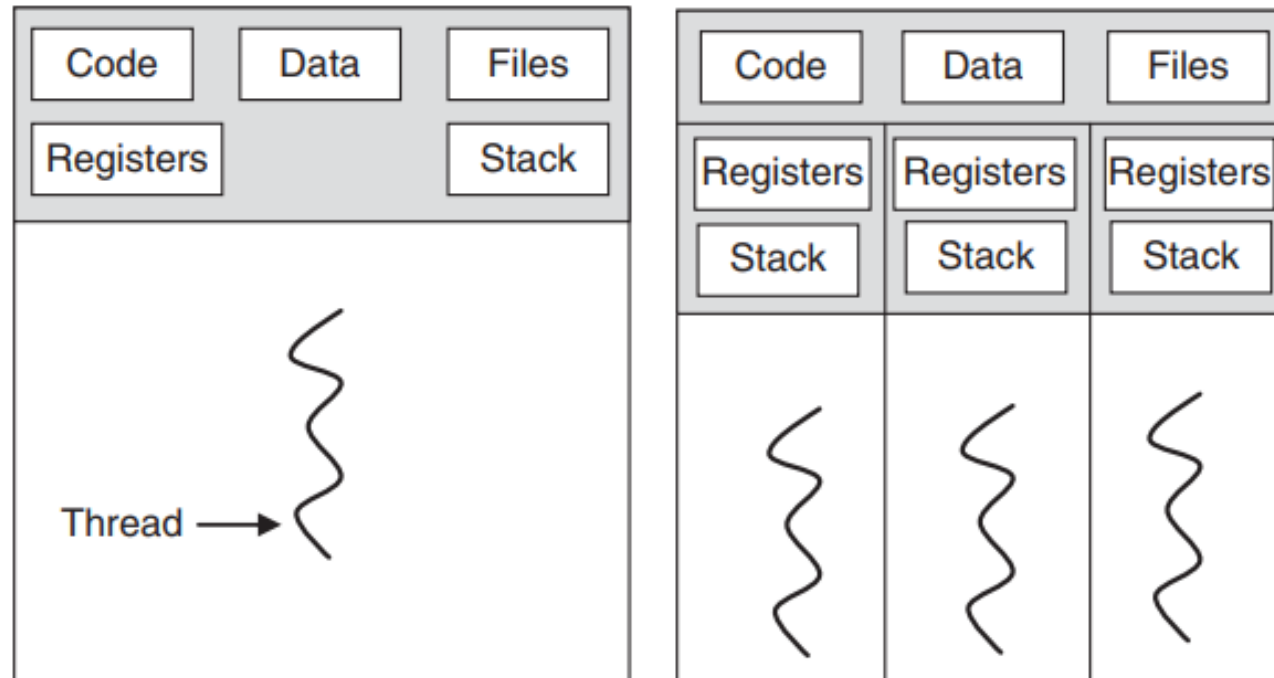
- **A thread** is a path of execution within a process and the basic unit to which the OS allocates processor time. **A process** can be *single-threaded* or *multi-threaded*.
- **Threads** are used for small tasks, whereas **processes** are used for more heavyweight tasks. Therefore, a thread is **often called** a **lightweight process**.
- **A thread** is the smallest unit of work managed independently by the scheduler of the OS.
- In RTOSs, the term tasks is often used for **threads** or **single-threaded processes**

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

- **Threads** use far less resources compared to **processes**. However, each thread of the **same process** has its own **thread status, program counter, registers, and stack**.



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

2) Memory Management

➤ Physical address

- Main memory is the most critical resource in a computer system **in terms of speed** at which programs run.
- Each memory location has a **physical address**. In most computer architectures, memory is byte-addressable, meaning that data can be accessed **8 bits at a time**, irrespective of the width of the data and address buses.
- In general, only system software such as **Basic Input/Output System (BIOS)** and **OS** can address physical memory.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

➤ Logic addresses

- Most application programs do not have knowledge of physical addresses. Instead, they use **logic addresses**.
- A logical address is the address at which a memory location appears to reside from the perspective of an executing application program.
- A logical address may **be different from** the physical address due to the operation of an address translator or mapping function.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

➤ Virtual memory

- In a computer supporting **virtual memory**, the term *physical address* is used mostly to differentiate from a *virtual address*.
- In particular, in computers utilizing a **memory management unit (MMU)** to translate memory addresses, the virtual and physical addresses refer to an address before and after translation performed by the MMU, respectively.
- **Why using the virtual memory ?**

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

➤ Protecting memory by Virtual memory

If two or more processes are **running at the same time** and use direct addresses, a memory **error** in one process could **destroy** the memory being used by the other process, taking down multiple programs due to a **single crash**.

→ The virtual memory technique, on the other hand, can ensure that each process is running in **its own dedicated address space**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

- **Memory allocation:** When a process is started, the **OS** **allocates memory** to the process and then loads the load module from disk into the memory allocated to the process.
- The simplest memory management technique is **single contiguous allocation**: all the memory is made available to a single application (*except the area reserved for the OS*).
- Another technique is **partitioned allocation** : It divides memory into multiple memory partitions; each partition can have **only one process**. → The memory manager selects a free partition and assigns it to a process when the process starts and deallocates it when the process is finished.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

- *Paged allocation*: divides memory into **fixed-size units** called **page frames** and the program's virtual address space into pages of the same size. The hardware **memory management unit (MMU)** maps pages to frames.
- The physical memory can be allocated on a page basis while the address space appears to be contiguous. *Paging* does not distinguish and protect programs and data separately.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

- *Segmented memory* allows programs and data to be broken up into logically independent address spaces and to aid sharing and protection.
- *Segments* are areas of memory that usually correspond to a logical grouping of information such as a code procedure or a data array.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

3) Interrupts Management

- An interrupt is a signal from a device attached to a computer or from a running process within the computer, indicating an event that needs *immediate attention*.
- The processor responds by suspending its current activity, saving its state, and executing a function called an **Interrupt Handler** (*Interrupt Service Routine, ISR*) to deal with the event.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

4) Multitasking

- Real-world events may occur simultaneously. Multitasking refers to the capability of an **OS** that supports **multiple independent programs** running on the same computer.
- It is mainly achieved through **time-sharing**, which means that each program uses a share of the computer's time to execute, which is addressed by **schedulers**, follow scheduling algorithms.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

5) File System Management

- Files are the fundamental abstraction of secondary storage devices. Each file is a named collection of data stored in a device.
- An important component of an OS is the **file system**, which provides capabilities of **file management**, auxiliary storage management, file access control, and integrity assurance.
- **File management** is concerned with providing the mechanisms for files to be stored, referenced, shared, and secured.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.1. General-Purpose Operating Systems

6) I/O Management

- Modern computers interact with a wide range of I/O devices: keyboards, mice, printers, disk drives, USB drives, monitors, networking adapters, and audio systems, ...
- One purpose of an OS is to **hide peculiarities** of hardware I/O devices **from the user**. The OS communicates with the I/O device through the **device controller** (is the primarily interface unit)
- I/O operations often have **high latencies**. Most of this latency is due to the slow speed of peripheral devices.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. *Real-Time Operating Systems - RTOS*

- Although a general-purpose OS provides a rich set of services it takes too much space and contains too many functions that may not be necessary for a **specific real-time application**.
 - Moreover, it is not configurable, and its inherent timing uncertainty offers no guarantee to **system response time**.
- Therefore, **a general-purpose OS is not suitable for Real-time embedded systems**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

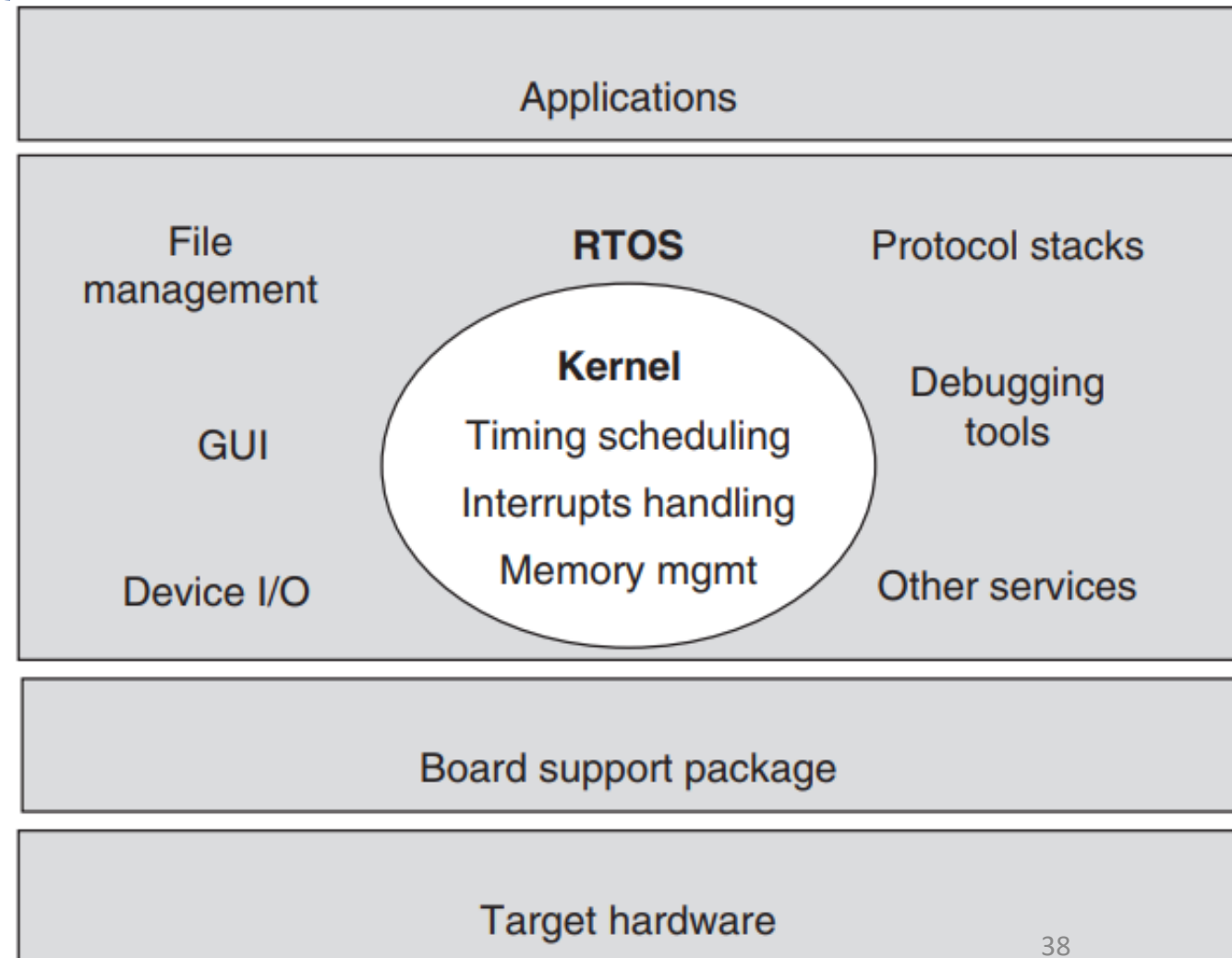
CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

+ An RTOS generally contains a **real-time kernel** (Figure) and other higher-level services: file management, protocol stacks, a Graphical User Interface (GUI), and other components.

→ Most additional services revolve around I/O devices.



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. *Real-Time Operating Systems*

- The heart of many computerized embedded systems is **real-time operating system**.
- An **RTOS** is an **OS** that supports the construction of applications that must *meet real-time constraints and obtain correct computation results*.
- A **real-time kernel** is software that manages the **time and resources** of a microprocessor or microcontroller and provides indispensable services such as task scheduling and interrupt handling to applications.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

There are three key requirements of **RTOS design**:

- ✓ The *timing behavior* of the OS *must be predictable*. For all services provided by the OS, the upper bound on the execution time *must be known*.
- ✓ The OS *must manage the timing and scheduling*, and the scheduler must be aware of *task deadlines*.
- ✓ The OS *must be fast*. A fast RTOS helps take care of soft real-time constraints of a system as well as guarantees hard deadlines

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

There are some most important Real-Time services for RTOS kernel:

1) Clocks and Timers:

- + Most embedded systems must keep track of the passage of time. The length of time is represented by **the number** of **system ticks** in most RTOS kernels.
- + The RTOS works by setting up a **hardware timer to interrupt** periodically, every millisecond, and bases all timings on the interrupts.
- + Based on the system **tick**, an RTOS kernel allows you **to call functions** of your choice after a given number of system ticks.
- + **Timers** improve the determinism of Real-Time applications and allow applications to set up events **at predefined intervals or time**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

2) Priority Scheduling:

+ Because real-time tasks have **soft** or **hard deadlines** , all tasks are not equal in terms of the urgency of getting executed. **Tasks** are typically prioritized in an RTOS.

+ To ensure that the **higher priority task** is executed **before** its deadline: if a higher priority task is released while the processor is serving a lower priority task → the RTOS should **temporarily suspend** the lower priority task and immediately schedule the higher priority on the processor for execution.

This process is called **preemption**.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

2) Priority Scheduling:

- + **Task scheduling** for real-time applications is typically priority-based and preemptive.
- + In priority-driven preemptive scheduling, the **preemptive scheduler** has a **clock interrupt task** that provides the scheduler with options to switch after the task has had a given period to execute.
- + The key to the performance of priority-driven scheduling is in **choosing priorities for tasks**. Priority-driven scheduling may cause low-priority tasks to starve and miss their deadlines.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3) Intertask Communication and Resource Sharing:

In an RTOS, **a task** cannot **call another task**. Instead, tasks exchange information through **message passing** or **memory sharing** and coordinate their execution and access to shared data using **real-time signals**, **mutex**, or **semaphore objects**,...

- **Real-time signals:** (*Signals are similar to software interrupts*)
 - + A signal is automatically sent to the parent when a child process terminates.
 - + Signals are also used for many other **synchronous and asynchronous notifications** (*such as waking a process when a wait call is performed and informing a process that it has issued a memory violation*).

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

- **Semaphores:** Semaphores are counters used for controlling access to resources **shared** among processes or threads. The value of a semaphore is the *number of units of the resource* that are *currently available*.

There are **two basic operations** on **semaphores**:

- (1) to *atomically increment* the counter.
 - (2) to *wait until* the counter is non-null and *atomically decrement* it.
- A semaphore tracks **only** how many resources are **free**; it does not keep track of which of the resources are free.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

- **Mutex:** It is used to allow multiple tasks to **share** the same resource, such as global data, but not simultaneously.
 - When resources are shared by multiple tasks, a *mutex object* is created with a unique name for each resource.
 - After that, any task that needs a shared resource must **lock** *the corresponding mutex object* from other tasks while it is using the resource.
 - The *mutex object* is set to **unlock** when *the data is no longer needed or the task is finished*.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

- Message Passing:

- + In addition to signals and semaphores, tasks can share data by sending messages in an organized message passing scheme.
- + Message passing can be direct or indirect.
- + Message passing can also be synchronous or asynchronous:

*In **synchronous message passing**, the sender process is blocked until the message primitive has been performed.*

*In **asynchronous message passing**, the sender process immediately gets control back.*

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

- Shared Memory:

+ Shared memory is a method that an RTOS uses to map **common physical space** into **independent process-specific virtual space**. Shared memory is commonly used to share information (resources) between **different processes** or **threads**.

+ Shared memory **must be accessed exclusively**, therefore, it is necessary to use mutex or semaphores to protect the memory area.

+ A **side effect** in using shared memory is that it may **cause priority inversion**, a situation that a low-priority task is running while a high-priority task is waiting.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

4) Asynchronous I/O: There are two types of I/O synchronization:

+ In *synchronous I/O*, when a user task requests the kernel for I/O operation and the requested is granted, the system will *wait until* the operation is completed *before* it can process other tasks.

→ *Applying* when the I/O operation is *fast* and *easy to implement*.

+ In *asynchronous I/O*, after a task requests I/O operation, while *this task is waiting for I/O to complete*, *other tasks that do not depend on the I/O results* will be scheduled for execution.

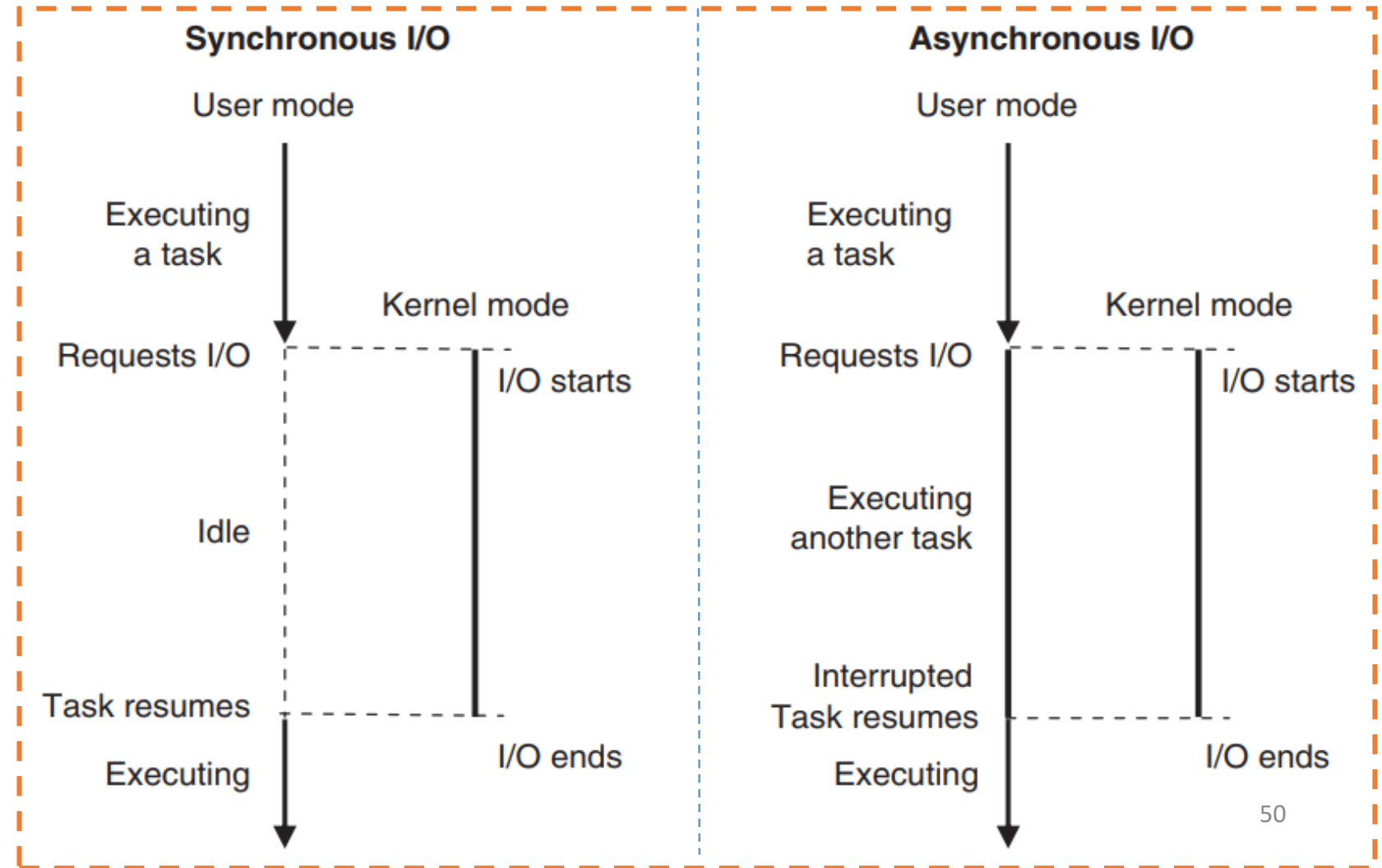
→ *To improve throughput, latency, and/or responsiveness.*

3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

4) Asynchronous I/O:



3. PHẦN MỀM HỆ THỐNG NHÚNG

3.2 Embedded Operating Systems

3.2.2. Real-Time Operating Systems

5) Memory Locking:

+ Memory locking is a real-time capability that is intended for a process to *avoid the latency* of fetching a page of memory. It is achieved by locking the memory so that the *page is memory-resident*.

→ *Remain* in the main memory and *allow* fine-grained control of which part of the application must stay in physical memory to *reduce the overhead associated*.

(For example, memory locking can be used to keep in memory *a thread* that monitors a critical process *that requires immediate attention*).

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

Middleware is software that has been abstracted out of the application layer, for a variety of reasons:

- ✓ **One reason** is that it may already be included as part of the off-the-shelf OS package.
- ✓ **Other reasons** to remove it from the application layer are:
 - + Allow *reusability* with other applications,
 - + *Decrease development costs or time* by purchasing it off-the-shelf-through a third party vendor,
 - + or *Simplify* application code.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

❑ Middleware

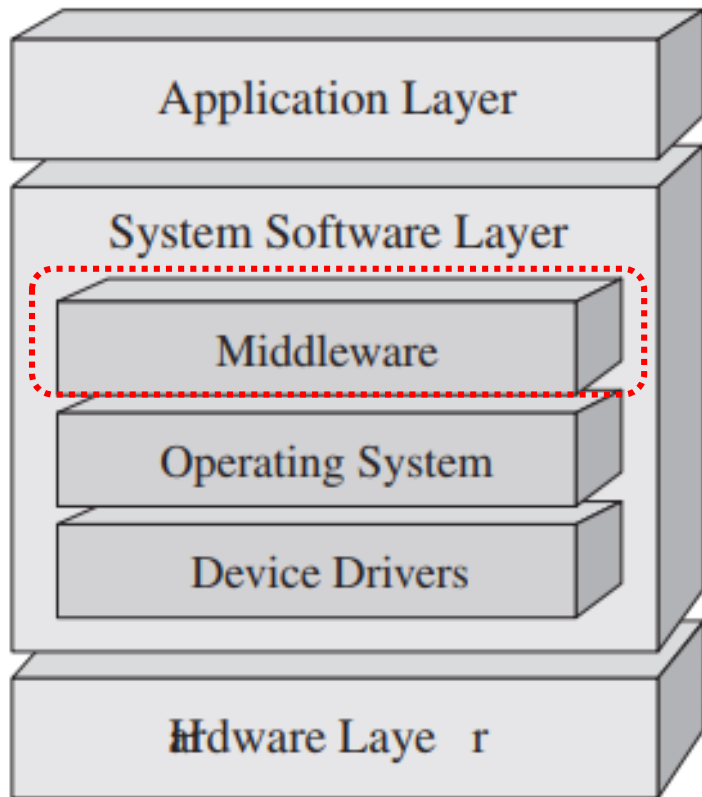
- + Middleware is any system software that is not the OS kernel, device drivers, or application software.
- + In an embedded system middleware is system software that typically sits on either the device drivers or on top of the OS, and can sometimes be incorporated within the OS itself. *(see Figure in next page)*

3. PHẦN MỀM HỆ THỐNG NHÚNG

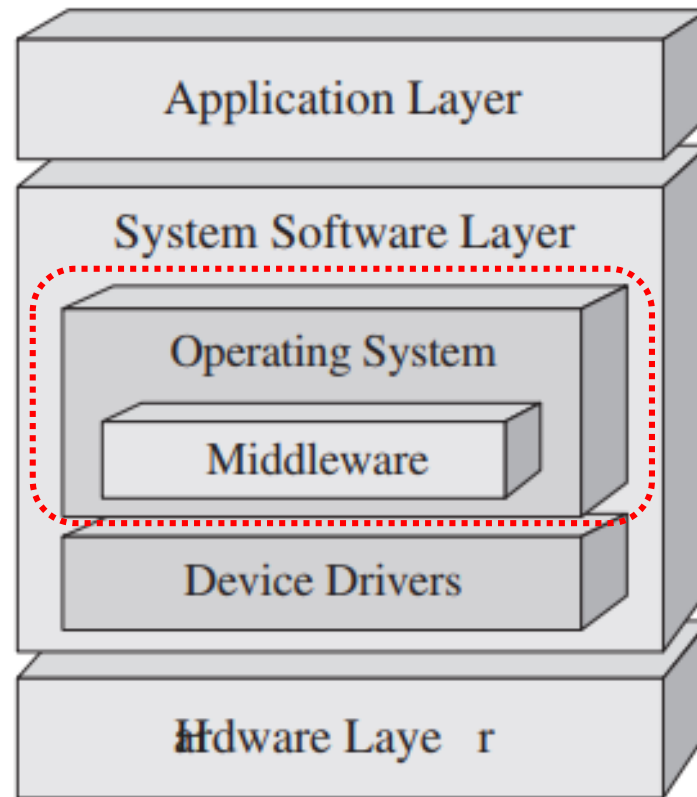
CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

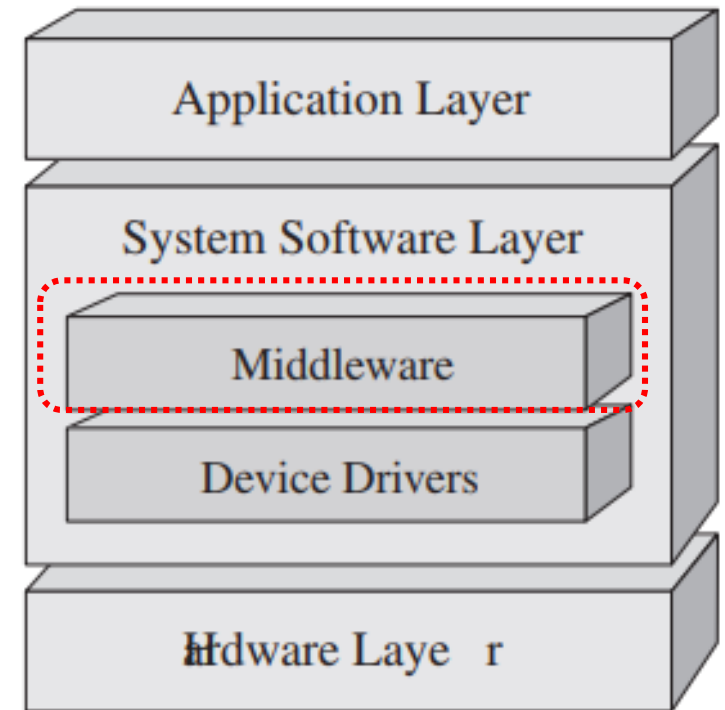
❑ Middleware



On top of OS



Incorporated within the OS itself



On Device Drivers

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

❑ Middleware

+ Middleware is usually the software **that mediates** between **application software** and the **kernel or device driver** software.

+ Middleware is also software **that mediates** and **serves** different application software.

(Specifically, middleware is an abstraction layer generally used on embedded devices **with two or more applications** in order to provide **flexibility, security, portability, connectivity, intercommunication, and/or interoperability mechanisms** between applications.

→ In short, **middleware** impacts the embedded system at all layers.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

There are some types of common middleware:

- 1) **General-purpose**, meaning they are typically implemented in a variety of devices, such as networking protocols *above the device driver layer* and *below the application layers* of the OSI model, file systems, or some virtual machines.
- 2) **Market-specific**, meaning they are unique to *a particular family of embedded systems* (such as *a digital TV standard-based software* that sits on an OS).

 - **Proprietary**, meaning it is *closed software* supported by a company that licenses it to others for use.
 - **Open**, meaning it is *standardized* by some industry committee and can be implemented and/or licensed by any interested party.

3. PHẦN MỀM HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

❑ Application Software

+ Application software sits *on top of the system software layer*, and is dependent on, managed, and run by the *system software*.

+ Application software is the software *within the application layer* that **gives the device** its character, differentiating it from other embedded devices.

→ Like embedded standards, **embedded applications** can be divided:

Market specific (Implemented in only a *specific type of device*, such as video-on-demand applications in *a interactive digital TV*).

General-purpose (Implemented across *various types of devices*).

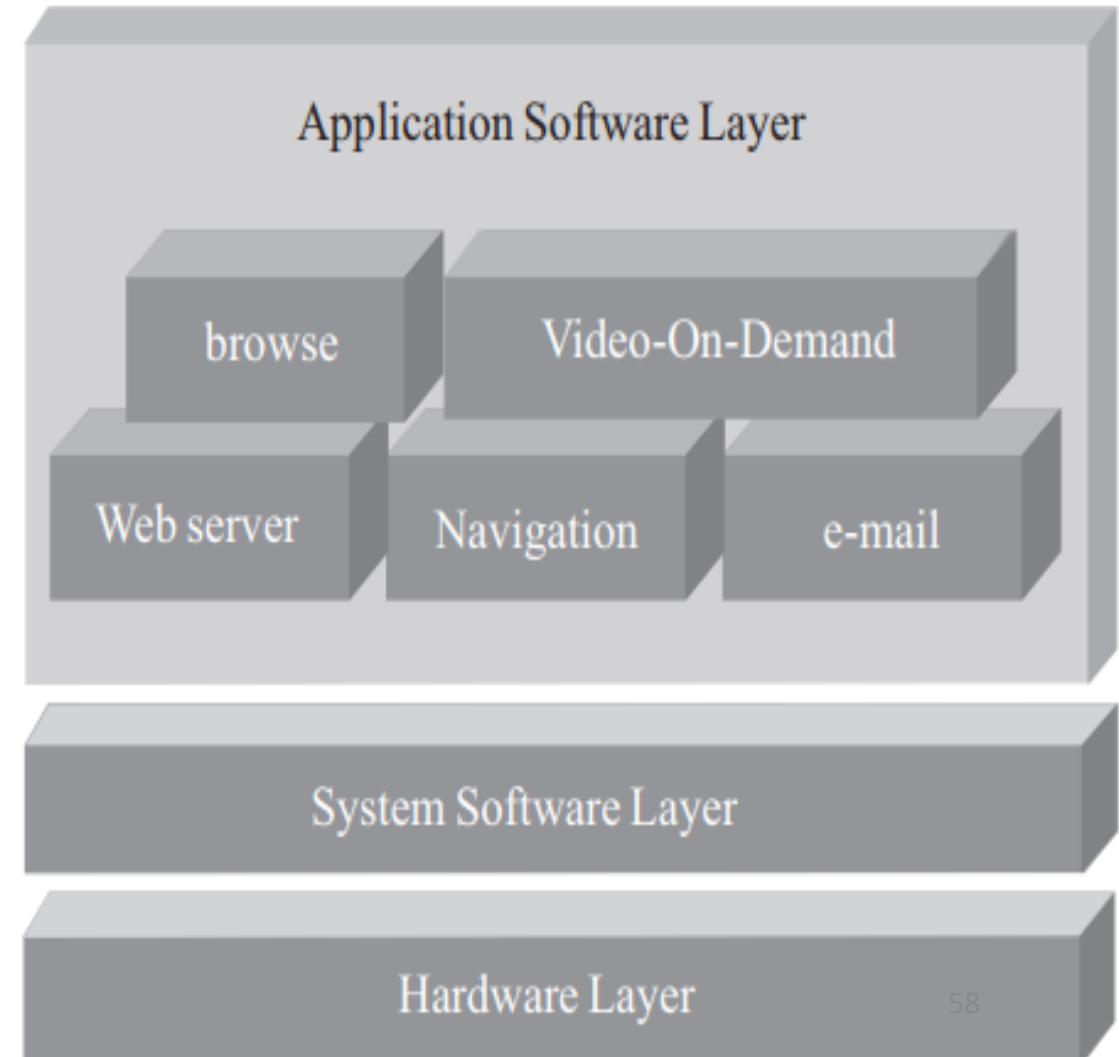
3. PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

3.3 Middleware and Application Software

❑ Application Software's Examples

- File Transfer Protocol Client Application ([browsers](#) or [Mp3 app](#),..).
- Simple Mail Transfer Protocol and E-Mail ([email](#)).
- Hypertext Transfer Protocol Client and Server ([Web server](#)).



CHƯƠNG 1: GIỚI THIỆU HỆ THỐNG NHÚNG

1. HỆ THỐNG NHÚNG
2. PHẦN CỨNG HỆ THỐNG NHÚNG
3. PHẦN MỀM HỆ THỐNG NHÚNG
4. CÔNG NGHỆ Multi-Core, SoC, and FPGA
5. MỘT SỐ ỨNG DỤNG HỆ THỐNG NHÚNG